



Markov Decision Process

Blink Sakulkueakulsuk

Lecture 1

Agent

Action

Environment

Reward

Next State



Rewards

It means, at time t , how good is the agent action

The ultimate goal of the agent is to maximize cumulative reward

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

This cumulative reward is called **return**

Values

We define the expected cumulative reward of a state s , the **value**.

$$\begin{aligned} v(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s] \end{aligned}$$

Action values

We can map value to both action and state

$$\begin{aligned} q(s, a) &= \mathbb{E}[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s, A_t = a] \end{aligned}$$

Markov decision processes

$$p(r, s \mid S_t, A_t) = p(r, s \mid H_t, A_t)$$

The equation above essentially means “the future state is independent of the past state given the present state”

MDP provides baseline for reinforcement learning.

Policy

Given a state, a policy $\pi(S)$ defines an agent's action

- Mapping from agent state to action

Deterministic policy: $\pi(S) = A$

Stochastic policy: $\pi(A|S) = p(A|S)$

Value function

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}[G_t \mid S_t = s, A_t \sim \pi(s)] \\ &= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t \sim \pi(s)] \end{aligned}$$

- $A_t \sim \pi(s)$ means that we choose action A from policy π
- $\gamma \in [0,1]$ is a discount factor

This is called a **Bellman equation**

If we want to find the optimal value, we can modify the equation to

$$v^*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) \mid S_t = s, A_t = a]$$

Model

A **model** predicts what will happen next in the environment

For example, a model P predicts the next state

$$P(s, a, s') \approx p(S_{t+1} = s' \mid S_t = s, A_t = a)$$

For example, a model R predicts the next reward

$$R(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

Exploration vs Exploitation

Exploration: gain knowledge

Exploitation: gain maximum reward

But do we have enough knowledge to know the action to gain maximum reward?

- It is a trade-off. We need to learn, but not too much.

Option 1: epsilon-greedy

A simple, yet powerful, algorithm

- $\epsilon \in [0,1]$ is a variable for exploration
- We select random action with probability ϵ
 - $a = \text{random}(A)$
- We select the best action with probability $1 - \epsilon$
 - $a = \text{argmax}_{a \in A} Q_t(a)$

Option 2: Upper Confidence Bounds

$$a_t = \operatorname{argmax}_{a \in A} Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}}$$

Basically, when we haven't explored some actions much, the value is high, and that action is selected and updated. Eventually, the value will converge to the true value for each action. Thus we can find an optimal action.

Lecture 1

Agent

Action

Environment

Reward

Next State



Agent

Action

Exploration &
Exploitation

Environment

Reward

Next State



Agent

Value Functions

Action

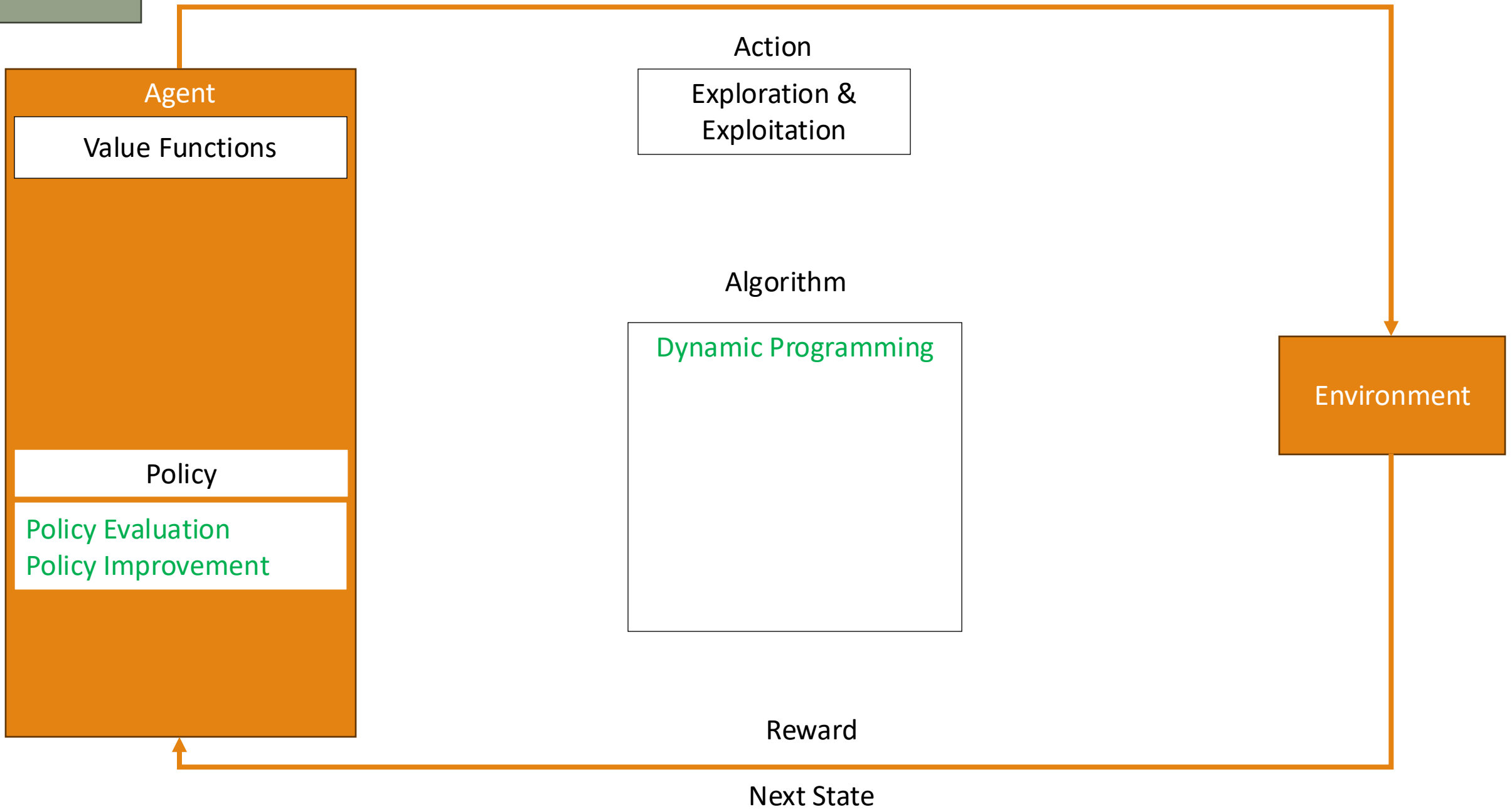
Exploration &
Exploitation

Environment

Reward

Next State





Markov Decision Process

This Lecture

Last time: Multi-arm bandit

- Single state problem with multiple actions
- No model

Today: Full sequential problem formulation

- Assume that true model is given
- Assume that the environment is fully observable

Next: Full sequential problem formulation

- What if true model is NOT given

Markov Decision Process

Markov decision processes (MDPs) tells us formally about the environment

“The future is independent of the past given the present”

MDPs can be used to formalize many RL applications

- Continuous MDPs (optimal control)
- Partially observable problems
- One state MDPs (bandits)

Markov Decision Process

MDPs can be formalized as a tuple $\{S, A, p, \gamma\}$

- S is the set of all possible states
- A is the set of all possible actions
- $p(r, s' | s, a)$ is the joint probability of a reward r and the next state s' , given a state s and an action a
- $\gamma \in [0, 1]$ is a discount factor

Noted that p defines the dynamics of the problem

Markov Decision Process

We can use $p(r, s' | s, a)$ to find state transition

$$p(s' | s, a) = \sum_r p(r, s' | s, a)$$

Or to find expected reward

$$\mathbb{E}[R | s, a] = \sum_r r \sum_{s'} p(r, s' | s, a)$$

To code this, we just loop through the states and rewards

Markov Property

“The future is independent of the past given the present”

Consider a sequence of random variables, $\{S_t\}_{t \in \mathbb{N}}$, indexed by time. A state s has the Markov property when

$\forall s' \in S,$

$$p(S_{t+1} = s' | S_t = s) = p(S_{t+1} = s' | h_{t-1}, S_t = s)$$

For all possible histories $h_{t-1} = \{S_1, \dots, S_{t-1}, A_1, \dots, A_{t-1}, R_1, \dots, R_{t-1}\}$

Example: cleaning robot

Given a cleaning robot

- It has two states: high and low battery
- It has two actions in high state: wait, and search
- It has three actions in low state: wait, search, and recharge
- Transition probability are described as followed
 - $p(S_{t+1} = \text{high} \mid S_t = \text{high}, A_t = \text{search}) = \alpha$
 - $p(S_{t+1} = \text{low} \mid S_t = \text{high}, A_t = \text{search}) = 1 - \alpha$

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	r_{wait}
low	wait	high	0	r_{wait}
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	0

Cleaning Robot MDPs

$$MDP = \{S, A, p, \gamma\}$$

$$S = \{high, low\}$$

$$A = \{search, wait, recharge\}$$

p = this look up table

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	r_{wait}
low	wait	high	0	r_{wait}
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	0

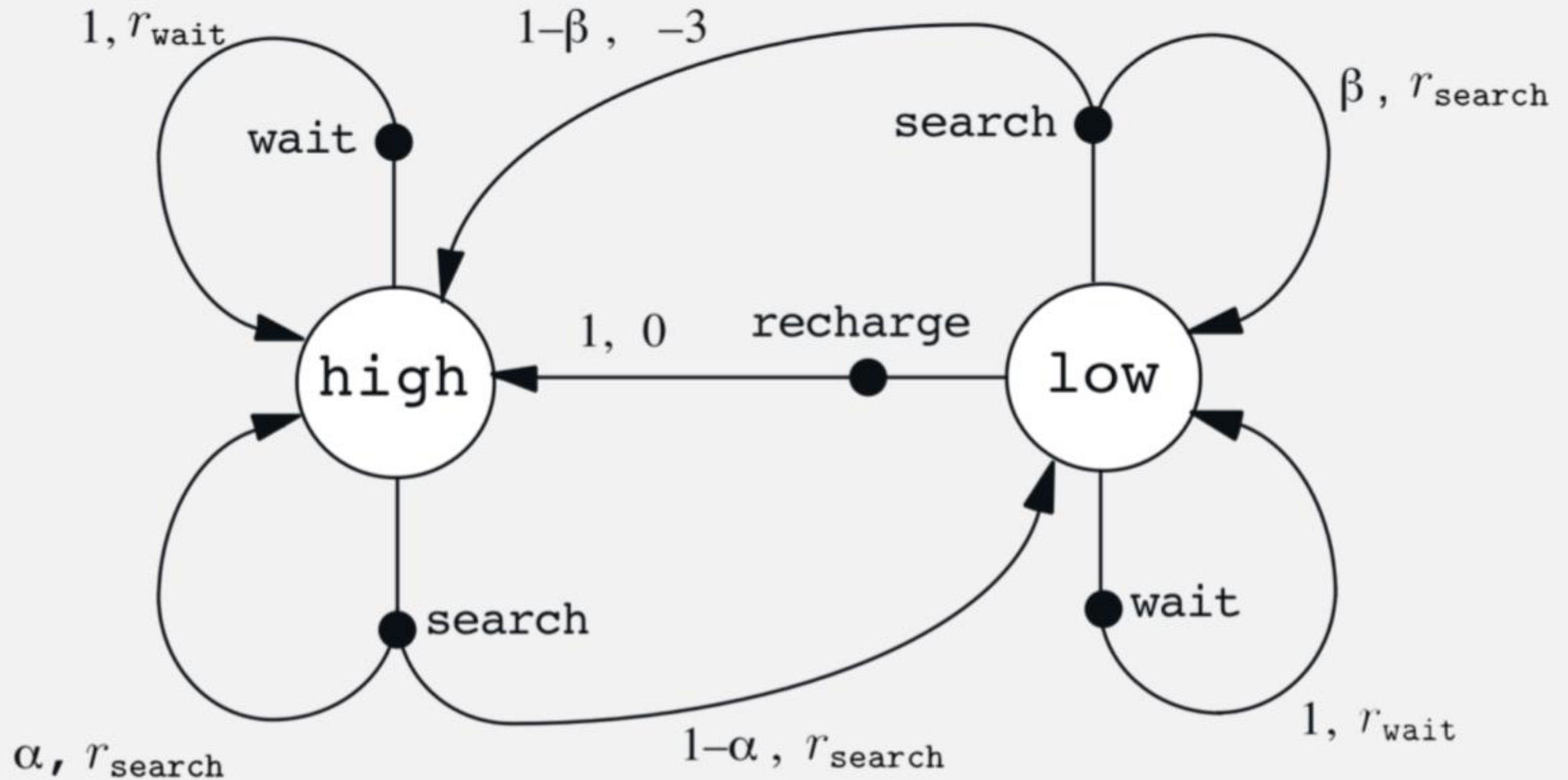
Let's go one more step further. When we talk about the value function, we need to find the expected reward. So, let's find some!

$$q(s = high, a = search) = \mathbb{E}[R | s = high, a = search]$$

$$q(s = high, a = search) = \alpha r_{search} + (1 - \alpha) r_{search} = r_{search}$$

$$q(s = low, a = search) = \mathbb{E}[R | s = low, a = search]$$

$$q(s = low, a = search) = (1 - \beta)(-3) + \beta r_{search} = (r_{search} + 3)\beta - 3$$



Return Revisited

Recall that

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- $\gamma \in [0,1]$ is a discount factor
- The discount represents the trade-off between short-term reward and long-term planning
- We can set the upper bound of the sum to T , the horizon of the problem, to bound the problem

Policy Revisited

Remind that

The Goal of an RL agent

Is to find a behavior policy that maximizes the expected return

A policy is a mapping $\pi: S \times A \Rightarrow [0,1]$

- It means that, for all states, and all action, there is a probability of taking an action on that state

Value function revisited

The value function denotes the expected return of state s

$$v_{\pi}(s) = \mathbb{E}[G_t \mid S_t = s, \pi] = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t \sim \pi(s)]$$

The equation above (hard to code) can be rewritten as:

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_r \sum_{s'} p(r, s' | s, a) (r + \gamma v_{\pi}(s'))$$

Action Value revisited

We can apply the same concept to action value equation

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}[G_t \mid S_t = s, A_t = a, \pi] \\ &= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

The equation above (hard to code) can be rewritten as:

$$q_{\pi}(s, a) = \sum_r \sum_{s'} p(r, s' \mid s, a) [r + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a')]$$

Note that

$$v_{\pi}(s) = \sum_a \pi(a \mid s) q_{\pi}(s, a) = \mathbb{E}[q_{\pi}(S_t, A_t \mid S_t = s, \pi)], \forall s$$

Optimal Value Function

The optimal value function is the maximum value of the value function of all policies

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

The optimal action value function is the maximum value of the action-value function of all policies

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

If we can find these value, we can find the best policy. Thus, **we solve the problem.**

Prediction and Control

Prediction

- A problem when we perform **policy evaluation**
- Estimating v_π or q_π

Control

- A problem when we perform **policy optimization**
- Estimating v^* or q^*

Four main Bellman Equations

Bellman Expectation Equation

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t \sim \pi(s)] \\q_{\pi}(s, a) &= \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]\end{aligned}$$

Bellman Optimality Equation

$$\begin{aligned}v^*(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) | S_t = s, A_t = a] \\q^*(s, a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a\right]\end{aligned}$$

Evaluating Policy

$$\pi \geq \pi' \leftrightarrow v_{\pi}(s) \geq v_{\pi'}(s) , \forall s$$

One policy is better than or equal to another **if and only** if its value function is greater or equal to another.

Optimal Policy

Theorem

For any Markov decision process

- There exists an optimal policy π^* that is better than or equal to all other policies, $\pi^* \geq \pi, \forall \pi$
- All optimal policies achieve the optimal value function,
$$v_{\pi^*}(s) = v^*(s)$$
- All optimal policies achieve the optimal action-value function,
$$q_{\pi^*}(s, a) = q^*(s, a)$$

Finding an Optimal Policy

We can use action value to find the optimal policy

Recall that there are many actions in one state. The action with highest action-value is the optimal action.

$$\pi^*(s, a) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in A} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

Noted that there can be multiple optimal policies

Solving the Bellman Optimality Equation

Using models (dynamic programming)

- Value iteration
- Policy iteration

Sampling Method

- Monte Carlo
- Q-learning
- Sarsa

Dynamic Programming

In general, set up a solution as a recursion, solve with iteration from the base case

In Reinforcement Learning, we iterate and update the value. There are two parts of the process.

- Policy evaluation
- Policy Improvement

BIG ASSUMPTION

We are doing a synchronous dynamic programming

- It means we update every state at the same time (kind of)

For an asynchronous dynamic programming, we can tweak the methods a little bit

Policy Evaluation

Given a policy, we want to estimate this

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | s, \pi]$$

We initialize all value to zero, and we iterate the equation above as an update

$$v_{k+1}(s) = \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | s, \pi] , \quad \forall s$$

Note that

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_r \sum_{s'} p(r, s' | s, a) (r + \gamma v_{\pi}(s'))$$

If $v_{k+1}(s) = v_k(s), \forall s$, we solve v_{π} .

Policy Evaluation



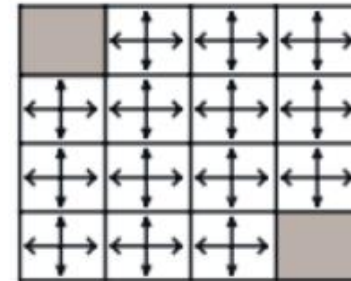
	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

Policy Evaluation

$k = 0$

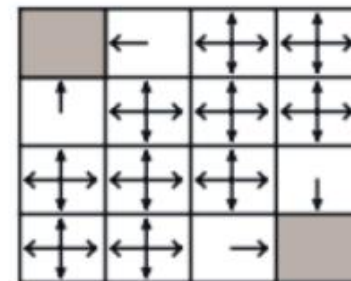
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0



random
policy

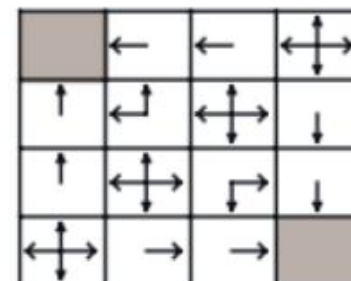
$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0



$k = 2$

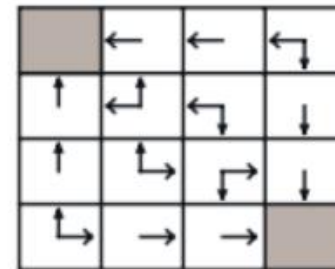
0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



Policy Evaluation

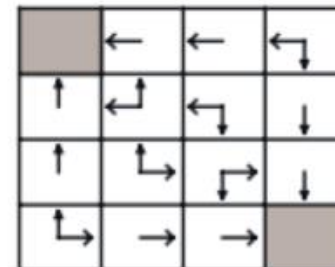
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



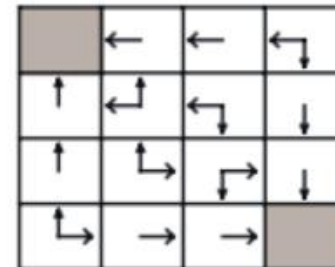
$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



optimal
policy

Policy Improvement

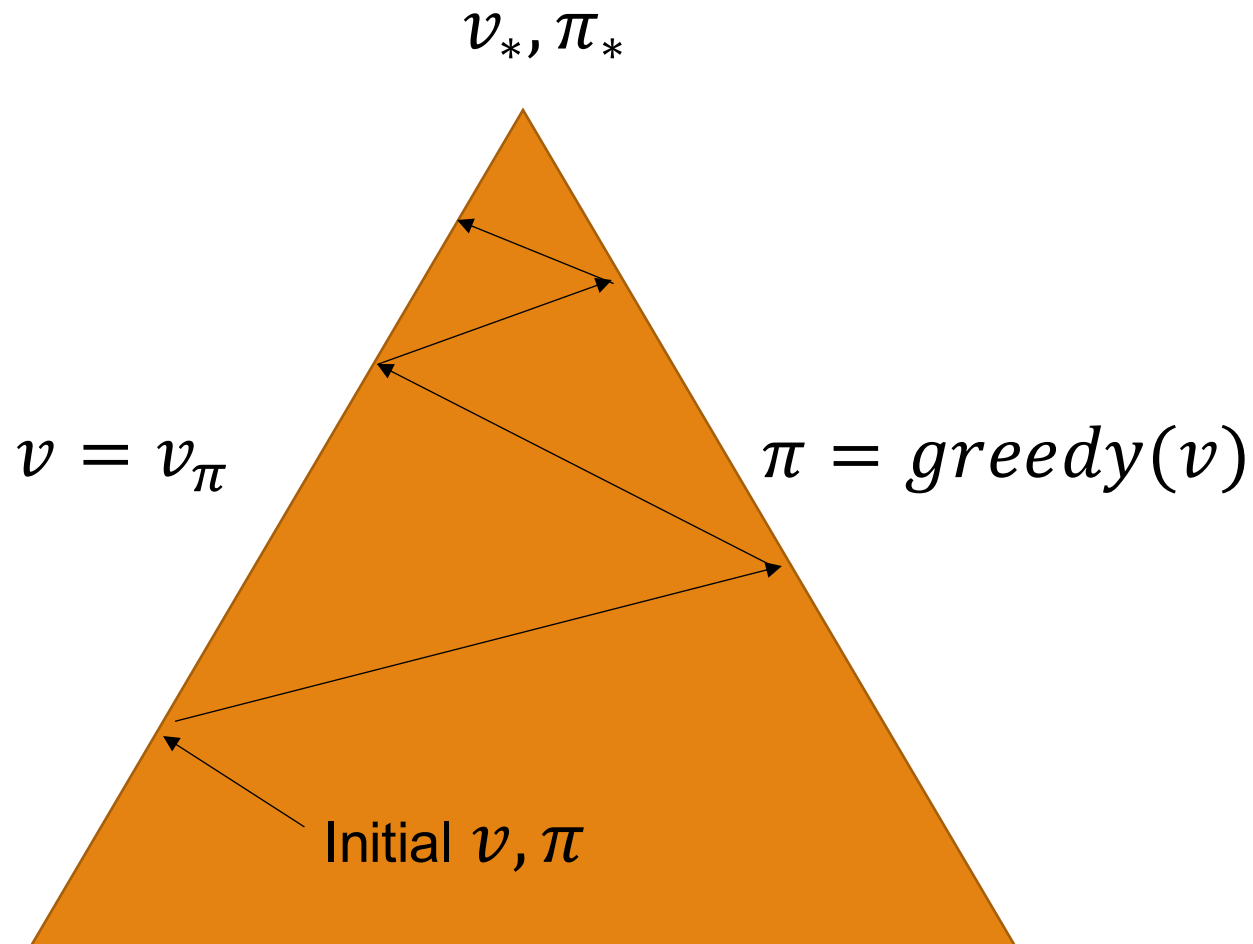
The example shows that we can improve a policy when we learn the value.

- In the example, we do not improve any policy

$$\begin{aligned}\forall s: \pi_{new}(s) &= \operatorname{argmax}_a q_{\pi}(s, a) \\ &= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t = a]\end{aligned}$$

We use the new policy to evaluate, and repeat.

Policy Iteration



Starting with initial v, π

Loop do:

Perform policy evaluation,

$$v = v_\pi$$

Perform policy improvement,

$$\pi' \geq \pi$$

Policy Improvement

$$\begin{aligned}\forall s: \pi_{new}(s) &= \operatorname{argmax}_a q_{\pi}(s, a) \\ &= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t = a]\end{aligned}$$

Since we improve the policy, $v_{\pi_{new}}(s) \geq v_{\pi}(s)$ for all s

It means that

$$v_{\pi_{new}}(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi_{new}}(S_{t+1}) | S_t = s]$$

Which is the Bellman optimality equation!

So, π_{new} must be either an improvement, or be optimal

Value Iteration

Or we can learn the policy on the fly

- We update the policy every iteration

We can take the Bellman optimality equation as an update

$$v_{k+1}(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a] \quad , \quad \forall s$$

It is the same as **policy iteration** with $k = 1$. Basically, we improve the policy every iteration.

Shortest Path Example

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

V_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

V_5

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

V_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

V_7

Summary

Prediction

- Bellman Expectation Equation
- Iterative Policy Evaluation

Control

- Bellman Expectation Equation + Greedy Policy Improvement
- Policy Iteration

Or

- Bellman Optimality Equation
- Value Iteration

Asynchronous Dynamic Programming

So far, we use **synchronous** dynamic programming

- We update all states in parallel at the same time (kind of)

However, we do not have to back up everything in parallel

- **Asynchronous Dynamic Programming** backs up states individually

Asynchronous Dynamic Programming

Three general approaches for asynchronous dynamic programming

- In-place dynamic programming
- Prioritized sweeping
- Real-time dynamic programming

In-place Dynamic Programming

During the updates, synchronous value iteration stores two copies of value function

$$\forall s \in S: v_{new}(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v_{old}(S_{t+1}) | S_t = s]$$
$$v_{old} \leftarrow v_{new}$$

However, we can store the value **in-place** if we do asynchronous dynamic programming

$$\forall s \in S: v(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) | S_t = s]$$

Prioritized Sweeping

We will pick and update states with a lot of Bellman error first

- Intuitively, such states have a lot to learn
- For example, we can use

$$\left| \max_a \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) | S_t = s] - v(s) \right|$$

We back up states with large Bellman error, then we update those states

Real-Time Dynamic Programming

Only update states that are relevant to the agent

For example, if an agent is in S_t , maybe we can just update that state value, and couple other relevant states

Full-Width Backups

Standard DP uses full-width backups

- It means that, to backup, the algorithm considers **all successor states and actions** using true transition model and reward function

DP is effective for medium-sized problem

- Million states
- Curse of Dimensionality can be a problem here